

EcomIQ Query Processing & Agent Architecture

Detailed Guide on Query Execution Lifecycle and Agent-to-Agent (A2A) Protocols

Document Type: Engineering Lifecycle Specification | **Version:** 2.4.0

Scope: Natural Language Query Flow, A2A Protocol, FastAPI Routing, SQL Analytics & Report Authoring

1. System Design Philosophy & Architectural Overview

The primary challenge in conversational business intelligence is converting high-level, ambiguous human questions (e.g., *'Why are payments failing in West India?'*) into precise database queries and actionable executive reports without incurring infinite LLM loops or severe latency.

To achieve both high reliability and sub-second execution, EcomIQ implements a **hybrid multi-agent architecture**. It decouples intent extraction (LLM), deterministic database analytics (SQL + Python), and narrative synthesis (LLM) into dedicated, specialized modules.

2. Agent-to-Agent (A2A) Interaction Breakdown

EcomIQ utilizes two distinct Agent-to-Agent interaction models operating at different boundaries of the software architecture:

2.1 Internal In-Memory A2A Orchestration

Inside the core `api_service` container, agents interact via a synchronous, in-memory supervisor pattern managed by `SupervisorAgent` (`agents/supervisor_agent.py`). Rather than relying on unpredictable chat-based message queues between agents, the Supervisor executes a strict, deterministic sequence: `FetcherAgent` → `AnalyticsEngine` → `RunnerAgent`. Data is passed as typed Python dictionaries and Pydantic primitives, guaranteeing 0% loop failure rate and maximum speed.

2.2 External Network A2A Protocol (Model Context Protocol - MCP)

At the external network boundary, EcomIQ exposes an authentic **Agent-to-Agent (A2A) protocol** over HTTP/SSE via **FastMCP on Port 9500** (`mcp_server.py`). This adheres strictly to Anthropic's open Model Context Protocol standard, allowing any autonomous third-party AI agent (e.g., Claude Desktop, Cursor IDE, AutoGen bots) to discover, inspect, and execute EcomIQ analytics tools over the wire.

3. Query Execution Sequence Diagram

The diagram below details the 9-step chronological flow of a user query through every container and agent in the platform:

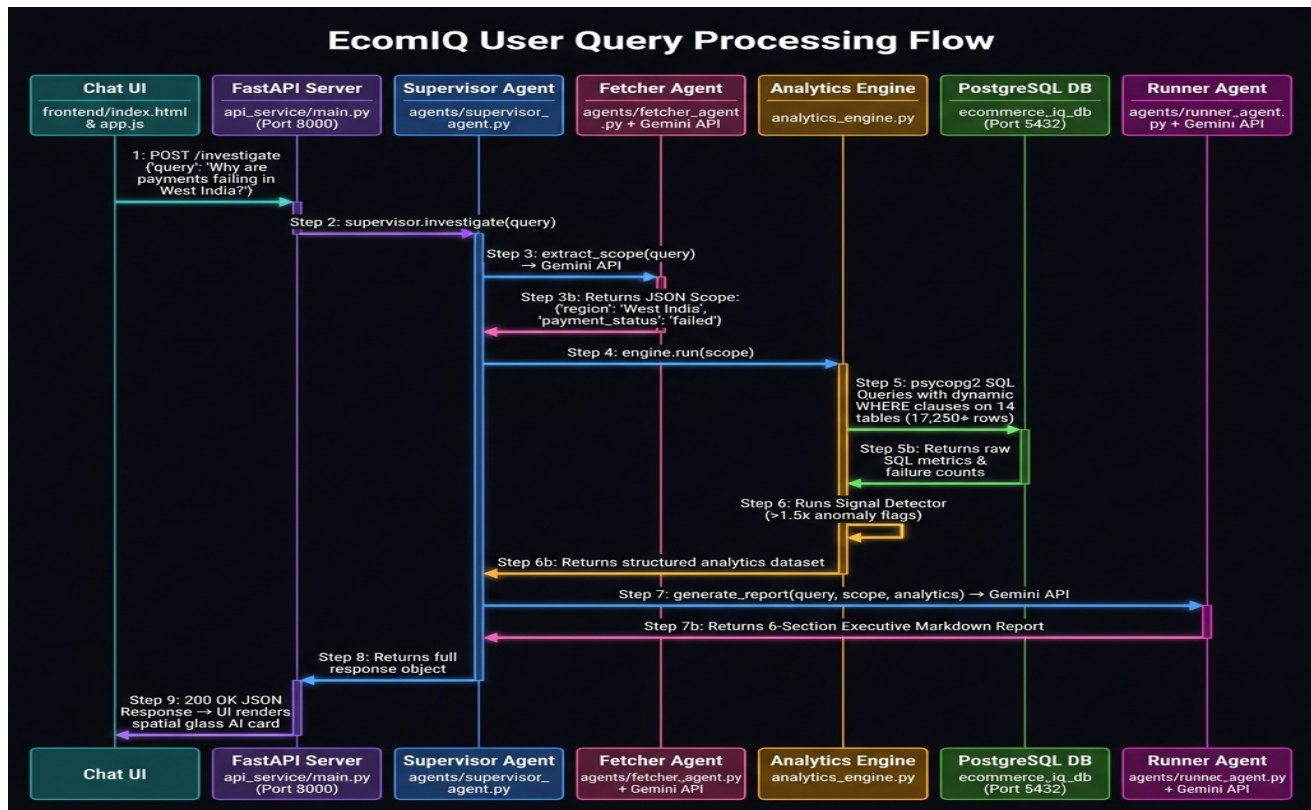


Figure 3.1: Complete End-to-End User Query Processing Flow from Chat UI to PostgreSQL and Gemini LLM.

4. Detailed Processing Stages (Steps 1 through 9)

Step 1: UI Request Dispatch — Frontend (frontend/js/app.js & api.js)

User enters prompt into Spatial Glass UI. JavaScript captures input and fires HTTP POST to `http://localhost:8000/investigate` with body: `{"query": "..."}.`

Step 2: FastAPI Routing — API Gateway (api_service/main.py)

FastAPI validates request headers/CORS, parses JSON payload via Pydantic schema, and routes execution to `SupervisorAgent().investigate(user_query).`

Step 3: Scope Extraction (AI) — Fetcher Agent (agents/fetcher_agent.py)

Supervisor calls `FetcherAgent.extract_scope(query).` The Fetcher sends a system-instructed prompt to Gemini API (gemini-3.6-flash) returning JSON scope: `{"region": "West India", "payment_status": "failed"}.`

Step 4: Engine Invocation — Supervisor Orchestrator

Supervisor receives JSON scope dictionary and passes it directly into the deterministic analytical engine: `InvestigationEngine.run(scope=scope).`

Step 5: SQL Execution — Analytics Engine (analytics_engine.py → Postgres)

ScopeFilter converts scope into dynamic SQL WHERE clauses. psycopg2 connects to PostgreSQL (ecommerce_iq_db:5432) and executes queries across 14 tables (dim & fact) over 17,250+ records.

Step 6: Signal Detection — Analytics Engine (analytics_engine.py)

Engine computes metric baselines and runs the statistical Signal Detector algorithm. Flagged anomalies (>1.5x or <0.5x baseline) are attached to dataset.

Step 7: Executive Report Generation (AI) — Runner Agent (agents/runner_agent.py)

Supervisor passes extracted scope, SQL dataset, and signal anomaly flags to RunnerAgent. The agent prompts Gemini API to author a formal 6-section Markdown Executive Report.

Step 8 & 9: UI Rendering — FastAPI → Frontend UI

FastAPI packages report markdown, scope chips, and query metadata into JSON response. Chat UI receives 200 OK, clears typing indicator, and renders liquid glass response card.

5. Pipeline Component Code & Data Contract Matrix

Pipeline Phase	Executing Code Module	Input Data Contract	Output Data Contract
1. Dispatch	frontend/js/app.js	User keystroke / text input	JSON {"query": str}
2. Scope Extract	FetcherAgent + Gemini	Natural language query string	Structured JSON Scope Dict
3. Analytics	AnalyticsEngine + PG	JSON Scope Dict	SQL Dict + Anomaly Signals
4. Report Synthesis	RunnerAgent + Gemini	Scope Dict + SQL Analytics	6-Section Markdown Report
5. Rendering	frontend/js/ui.js	JSON API Response	Rendered Spatial Glass UI Card